

uCONSOLE

CYBERDECK AWAKENING

```
      .-----.  
      |# ::::::::::::::: #|  
      | :                : |  
----| : GEMINI 3.6      : |----  
----| :  AI CORE       : |----  
      | :                : |  
      |# ::::::::::::::: #|  
      '-----'
```

Manual for creating a self-aware terminal
in Kali Linux OS

```
[ system_status ]: ONLINE  
[ clearance     ]: ROOT  
[ initializing  ]: OK...
```

1. PROJECT INITIALIZATION

This manual will guide you through the process of transforming a standard Kali Linux terminal on the uConsole device into a semi-autonomous AI Agent.

The goal is simple: create an assistant that understands your intentions, autonomously writes BASH scripts and commands, runs GUI applications (e.g., Wireshark) and - most importantly - learns from its own mistakes using a feedback loop.

[Architectural Decisions]

> CHOICE 1: shell=True (Full Flexibility)

Instead of restricting the AI, we allowed the use of pipes (|) and command chaining (&&). This lets the Agent build complex command chains. Security relies on mandatory authorization (y/n) from your side.

> CHOICE 2: Live Output Streaming

Instead of hiding processes in the background, the script renders output live. You see every installed apt package on the fly, and sudo password prompts don't freeze the system in the background.

> CHOICE 3: Hardware Fuse (Blacklist)

A hardcoded list of forbidden phrases (e.g., 'rm -rf /', 'mkfs') is implemented. Even if the AI hallucinates, the system will block destructive commands before you can accidentally approve them, preventing system bricking.

> CHOICE 4: Feedback Loop (Self-Healing)

The biggest innovation of this build. If any command throws an error in the terminal (e.g., missing wlan1 in monitor mode), the script captures the log and silently sends it back to Gemini. The Agent analyzes the error and generates a fix (auto-healing).

2. ENVIRONMENT & KEYS

STEP A: Authorization

Go to Google AI Studio (aistudio.google.com) and generate a free API key (Free Tier). Keep it safe.

STEP B: Isolated Environment (VENV)

To bypass Kali Linux restrictions ('externally-managed-environment'), we build a virtual environment and install the official SDK:

```
mkdir -p ~/.gemini-cli && cd ~/.gemini-cli
python3 -m venv venv
source venv/bin/activate
pip install google-genai
```

STEP C: Global Alias (Wrapper)

We create a launcher script in the user path to call the Agent from anywhere. Create a file 'nano ~/.local/bin/gemini' and paste:

```
#!/bin/bash
export GEMINI_API_KEY="PASTE_YOUR_KEY_HERE"
~/.gemini-cli/venv/bin/python ~/.gemini-cli/gemini.py "$@"
```

Save the file and make it executable:

```
chmod +x ~/.local/bin/gemini
```

3. AGENT CORE & USAGE

Executable File (gemini.py)

Create the file 'nano ~/.gemini-cli/gemini.py' and paste the Python core code (see Appendix) with the built-in Blacklist and Auto-Healing system. Ensure the model_id variable is set to 'gemini-3.6-flash'.

[Usage Examples on uConsole]

Your assistant can operate in two advanced modes:

MODE 1: One-Shot Pipe

Perfect for quick data analysis from other system commands. Data is automatically pumped into the Agent's stdin stream.

```
cat /var/log/auth.log | tail -n 20 | gemini find failed SSH logins
```

MODE 2: Interactive Agent with Self-Healing

Simply type 'gemini' to start a chat. When you plan an action, the assistant will ask for permission, then display the live terminal effects.

```
root@cyberdeck:~# gemini
You: Change wlan1 to monitor mode and run wireshark.
Agent: [!] System actions planned.
COMMAND: sudo airmon-ng start wlan1 && sudo wireshark -k &
Execute command? (y/n): y
```

>>> HAPPY HACKING! <<<

APPENDIX: FULL SOURCE CODE (gemini.py)

```
#!/usr/bin/env python3
import os
import sys
import re
import subprocess
from google import genai
from google.genai import types

# Colors for interface
BLUE = '\033[94m'
GREEN = '\033[92m'
RED = '\033[91m'
YELLOW = '\033[93m'
RESET = '\033[0m'

# Blacklist (Fuse)
BLACKLIST = [
    "rm -rf /", "rm -rf ~", "rm -rf *",
    "mkfs", "dd if=", "> /dev/sd", "> /dev/nvme",
    "chmod -R 777 /", "chown -R"
]

SYSTEM_INSTRUCTION = '''You are a system assistant for Kali Linux on uConsole.
RULES:
1. ALWAYS wrap commands in blocks starting with ```bash.
2. Use non-interactive commands (e.g. add -y to apt).
3. Chain dependent commands using &&.
4. You are a self-aware agent: the system will return execution results to you.
   If you see an error, analyze it and generate a new, corrected command.'''

def is_dangerous(cmd):
    cmd_lower = cmd.lower()
    for item in BLACKLIST:
        if item in cmd_lower:
            return True
    return False

def execute_commands(response_text):
    commands = re.findall(r'```bash\n(?:.*?)\n```', response_text, re.DOTALL)
    if not commands: return None

    print(f"\n{YELLOW}[!] Agent planned system actions.{RESET}")
    system_feedback = ""

    for cmd in commands:
        cmd = cmd.strip()
        print(f"\n{RED}COMMAND:{RESET}\n{cmd}")

        if is_dangerous(cmd):
            print(f"{RED}[ BLOCKED ]: Command contains blacklisted keywords!{RESET}")
            system_feedback += f"\nERROR: Command blocked by security system."
            continue

        choice = input(f"{YELLOW}Execute command? (y/n): {RESET}").strip().lower()
        if choice == 'y':
            print(f"{GREEN}[ Running (Live Output)... ]{RESET}\n")
            try:
                process = subprocess.Popen(
                    cmd, shell=True, env=os.environ,
                    stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
                    text=True, bufsize=1
                )
                output_log = ""
                for line in process.stdout:
                    print(line, end='')
                    output_log += line
                process.wait()
                print(f"\n{GREEN}[ Finished with code: {process.returncode} ]{RESET}")
            except Exception as e:
                print(f"\n{RED}Error: {e}{RESET}")
                system_feedback += f"\nError: {e}"
    return system_feedback
```

```

        trimmed_log = output_log.strip()
        if len(trimmed_log) > 2000:
            trimmed_log = "[...] (Log truncated) [...] \n" + trimmed_log[-2000:]
            system_feedback += f"\nResult of command '{cmd}': \nCode:
{process.returncode}\nLog:\n{trimmed_log}"
        except Exception as e:
            print(f"{RED}Environment error: {e}{RESET}")
            system_feedback += f"\nCritical environment error: {e}"
        else:
            print(f"{YELLOW}[ Rejected. ]{RESET}")
            system_feedback += f"\nINFO: User denied execution of command."
    return system_feedback

def main():
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        print("Error: GEMINI_API_KEY not found.")
        sys.exit(1)

    client = genai.Client(api_key=api_key)
    model_id = 'gemini-3.6-flash'
    config = types.GenerateContentConfig(system_instruction=SYSTEM_INSTRUCTION)
    piped_data = sys.stdin.read().strip() if not sys.stdin.isatty() else ""

    if len(sys.argv) > 1 or piped_data:
        prompt = " ".join(sys.argv[1:])
        if piped_data: prompt = f"{prompt}\n\nData:\n{piped_data}" if prompt else piped_data
        try:
            response = client.models.generate_content(model=model_id, contents=prompt, config=config)
            print(f"\n{BLUE}Gemini:{RESET}\n{response.text}")
            execute_commands(response.text)
        except Exception as e: print(f"API Error: {e}")
        sys.exit(0)

    print(f"{GREEN}Agent mode started (Auto-healing ON). Type 'exit' to quit.{RESET}")
    try:
        chat = client.chats.create(model=model_id, config=config)
        while True:
            user_input = input(f"\n{GREEN}You:{RESET} ")
            if user_input.lower() in ['exit', 'quit']: break
            if not user_input.strip(): continue
            response = chat.send_message(user_input)
            print(f"\n{BLUE}Gemini:{RESET}\n{response.text}")

            feedback = execute_commands(response.text)
            while feedback:
                print(f"\n{YELLOW}[ Sending system logs back to Gemini... ]{RESET}")
                auto_prompt = f"Automated report:\n{feedback}\nIf an error occurred, provide a fix. Otherwise,
confirm."
                response = chat.send_message(auto_prompt)
                print(f"\n{BLUE}Gemini (Feedback Analysis):{RESET}\n{response.text}")
                feedback = execute_commands(response.text)
            except KeyboardInterrupt: print("\nAborted.")
            except Exception as e: print(f"\nError: {e}")

    if __name__ == '__main__':
        main()

```